Quantum-inspired catalytic QUBO search for balanced hypergraph partitioning

Baijian Yao[1] and Xiwei Pan[2]

[1]Department of Electronic and Computer Engineering, Hong Kong University of Science and Technology, Hong Kong SAR, China.
[2]Thrust of Advanced Materials, The Hong Kong University of Science and Technology (Guangzhou), Guangzhou, China.

Abstract

Hypergraph partitioning seeks balanced blocks with few crossing nets. Classical multilevel methods provide strong solutions, while quantum approaches face encoding and search difficulties. Inspired by temporary quantum-annealing interactions, we modify the classical variational search path of an exact cut-net QUBO and restore its target energy before readout. Incidence-dependent catalysts yield a local response that predicts perturbation growth in the implemented update. On 30 hypergraphs, a preselected FP32 trajectory with conditional-QUBO refinement improves 20 direct KaHyPar references at exact balance, one at 1% imbalance and none at 3%, comparing each solver's best of three trials. These direct baselines do not establish superiority over stronger repaired classical pipelines. Native CUDA execution reduces latency, while time–quality curves retain residual quality gaps that delimit practical gains from the local mechanism.

Keywords: quantum-inspired optimization, QUBO, hypergraph partitioning, catalysts, free-energy machine

Introduction

Hypergraph partitioning allocates interacting tasks or components to balanced blocks while limiting dependencies between blocks. A net represents a signal or dependency involving many vertices; the cut-net objective charges it once when it crosses blocks. This model is used for circuit netlists and distributed computations[1,2]. Multilevel methods combine contraction and local refinement[3,4], and KaHyPar and its parallel successor provide strong classical references[5,6]. For an energy-based solver, the challenge is to preserve the many-vertex objective while creating useful motion toward a balanced partition. Encoding and dynamics answer different questions: what is minimized, and which states can be reached.

Quantum annealing searches through time-dependent Hamiltonians[7–9], while quantum approximate optimization algorithms (QAOA) use alternating parameterized circuits[10]. Ising formulations connect these methods to discrete optimization[11]. Quadratic unconstrained binary optimization (QUBO) uses two-body interactions. Exact quadratization represents many-body costs using auxiliary variables[12–14]. Graph partitioning has been tested on D-Wave hardware and through hybrid QUBO decomposition[15], while constraint-preserving mixers and drivers address fixed block sizes[16,17]. For a hypergraph, the energy must

Baijian Yao: byao@connect.ust.hk; Xiwei Pan: xiwei.pan@connect.hkust-gz.edu.cn

charge a crossing net once, even when many vertex pairs lie in different blocks. Summing pairwise cut costs can therefore change which partition is preferred.

Li et al. derive encodings for several hypergraph cut functions and test simulated QAOA and annealing on small 3-uniform inputs[18]. Their experiments show that the balance penalty strongly affects both cut quality and feasibility. Ibrahim et al. instead optimize distributions over partitions for fairness[19]. For cut-net minimization on real hypergraphs, two difficulties remain. A pairwise approximation can favor a partition with a larger native cut; an exact auxiliary QUBO preserves the optimum but need not guide a finite search toward a good feasible partition. In particular, increasing the balance penalty discourages changes in block size, while improving a balanced cut can require coordinated changes of several vertex labels. The practical question is how to enable these rearrangements while retaining the exact cut-net objective.

Quantum annealing suggests a way to modify this search process. Temporary catalyst interactions change the intermediate Hamiltonian and are removed before the final problem is read out. They can alter transitions or gaps[20–22], with effects depending on the model and schedule[23,24]. We adapt this principle to a classical variational search of the exact QUBO. The intended role of the catalyst is to encourage coordinated changes in vertex probabilities before they settle into a partition. The hypergraph specifies which changes should be coupled: shared nets define local interactions, while the incidence structure supplies collective directions that redistribute probability between blocks without changing their mean loads. Adding these interactions temporarily changes the intermediate free energy; removing them restores the original QUBO for the final stage and evaluation.

We implement this construction with a classical free-energy machine (FEM)[25]. Exact examples distinguish objective distortion from the difficulty of searching an exact encoding. A reduced free-energy Hessian then relates the temporary interactions to the growth of balanced fluctuations, providing a mechanism to test on original hypergraphs. On 30 original hypergraphs, we evaluate a single preselected FP32 trajectory and explicit conditional-QUBO refinement against direct KaHyPar runs at 0%, 1% and 3% allowed imbalance. Historical controls separately isolate block-label symmetry and catalyst components. The comparisons distinguish local dynamical predictions, returned partition quality and time to a specified target; cached preprocessing and calibration costs delimit their practical interpretation.

Results

Faithful hypergraph cut-net objective in QUBO

Two-way hypergraph partitioning divides a vertex set into two disjoint blocks, subject to a capacity bound, while minimizing the weight of nets that cross between them[5,18]. A graph joins pairs of vertices; a hypergraph $\mathcal{H} = (V, \mathcal{E})$ permits a net $e \in \mathcal{E}$ to contain any number of vertices. We write $n = |V|$, $m = |\mathcal{E}|$, $r_e = |e|$ and $w_e \geq 0$ for its weight. We first ask whether a binary energy preserves this partitioning objective.

Binary $x_i \in \{0, 1\}$ records the block of vertex i. For a graph with edge set E_2, the weighted edge-cut cost is

$$E_{\mathrm{G}}(\mathbf{x}) = \sum_{\{i,j\} \in E_2} w_{ij}(x_i - x_j)^2. \tag{1}$$

Each edge contributes its weight when its endpoints occupy different blocks. For a hypergraph, $k_e = \sum_{i \in e} x_i$ counts the vertices of net e assigned to one block. The cut-net objective is

$$E_{\mathrm{cut}}(\mathbf{x}) = \sum_{e} w_e \mathbb{I}(0 < k_e < r_e). \tag{2}$$

A crossing net is charged once, irrespective of the number of separated vertex pairs. For example, a four-vertex net split 1:3 or 2:2 costs one net weight in either case; a pair surrogate instead counts three or four crossing pairs. For two-vertex nets, cut-net and graph cut coincide.

The unit-load two-way problem also imposes balance: strict capacity permits $N = \sum_i x_i \in \{b, n - b\}$, with $b = \lfloor n/2 \rfloor$. Fixing the block labels so that $N = b$ gives an equivalent oriented problem, used in the original search policy. An allowed imbalance band specifies integer bounds on N. All reported cuts are scored on the original nets and accompanied by their actual imbalance. An

allowed imbalance is an upper bound; a more balanced output remains feasible at that setting.

The first obstacle is objective distortion. We compare two quadratic clique surrogates that sum $w_e k_e(r_e - k_e)$ or divide that quantity by $r_e - 1$ for non-singleton nets; singleton nets contribute zero. Both sum costs for vertex pairs separated within a net. For a net of size four, a 1:3 split separates three pairs while a 2:2 split separates four, although both cut the same single net. Normalizing these pair costs by a common net-size factor retains that distinction. The resulting surrogate can therefore prefer a different partition even when it is minimized exactly.

Exact enumeration gives concrete counterexamples (Fig. 1). Each has 12 vertices, ten mixed-size nets and 924 balanced assignments. In example A, the native optimum is 7, but every minimum of the unit-pair surrogate has native cut 8. In example B, the native optimum is 5; the normalized surrogate selects native cut 6, while the unit-pair surrogate has minimum-state cuts ranging from 8 to 10. These are illustrative counterexamples, not estimates of average surrogate performance.

An auxiliary QUBO avoids this objective distortion by representing the net indicator exactly. QUBO expresses an energy as $\mathbf{y}^\mathsf{T}\mathbf{Q}\mathbf{y} + c_0$ for binary $\mathbf{y}$[11,12]; additional bits can represent higher-order costs through quadratic interactions[13]. Using two bits u_e, v_e per net, our strict-balance energy is

$$E_{\mathrm{QUBO}} = \sum_e w_e\big[1 + (r_e - 1 - k_e)u_e + (k_e - 1)v_e\big] + \lambda(N - b)^2. \tag{3}$$

The net sum defines E_{net} and uses $n + 2m$ binary variables including the vertices. At fixed vertices, its auxiliary minimum is the native cut:

$$\min_{u_e, v_e \in \{0,1\}} w_e[1 + (r_e - 1 - k_e)u_e + (k_e - 1)v_e] = w_e \mathbb{I}(0 < k_e < r_e). \tag{4}$$

Indeed, $k_e = 0$, $0 < k_e < r_e$ and $k_e = r_e$ give minimizing costs 0, w_e and 0, respectively, including zero for singleton nets. The encoded cost therefore has no preference for a net's split ratio. On both examples, auxiliary minimization reproduces the native cut on all 924 balanced assignments,

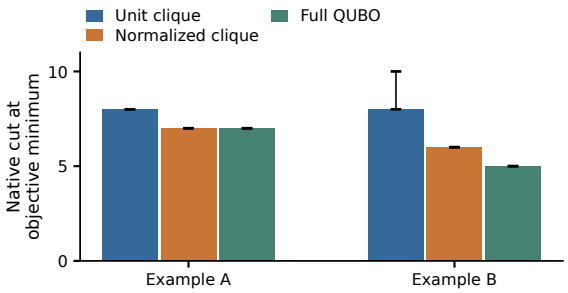


Figure 1 Preserving the cut-net optimum; Native cut among global minima of each objective on two exact counterexamples; Bar heights use the best native cut among tied minima; the whisker on example B's unit clique spans all native cuts among its minima, not an uncertainty interval; The complete auxiliary QUBO recovers the native optima after auxiliary minimization

including every optimum. The exact construction removes the discrepancy of the clique surrogates; Methods specifies the capacity-penalty bound and allowed imbalance.

Exact quadratic encoding resolves objective distortion but leaves a second difficulty in the search itself (Table 1). The original oriented balance penalty must be strong enough to exclude infeasible global minima. When $\lambda > D_{\max}$, flipping one vertex from a balanced state incurs a penalty greater than its largest possible cut reduction; $D_{\max}$ is the maximum weighted incidence degree. If the auxiliaries are already minimized, none of their individual flips helps either. Every such balanced state is therefore a one-bit local minimum, regardless of cut quality. For a feasible state, the change from flipping vertex i obeys

$$\Delta_i E_{\mathrm{QUBO}} \geq \lambda - D_i > 0, \qquad D_i = \sum_{e \ni i} w_e. \tag{5}$$

This coordinate barrier motivates coordinated changes during the search. Supplementary Note 2 gives the proof and distinguishes the slack and complement-symmetric constraints.

Temporary catalysts change solver dynamics

The coordinate barrier motivates changing the search before vertex labels become binary. To analyse this possibility, we describe uncertain labels by independent probabilities $p_i = \Pr(x_i =$

Table 1 Cut-net encodings

Construction	Order	Binary variables	Property relevant to cut-net search
Unit-pair clique	2	n	Different objective; example A selects cut 8 instead of 7.
Normalized clique	2	n	Different objective; example B selects cut 6 instead of 5.
Complete auxiliary QUBO	2	$n + 2m$	Exact cost after auxiliary minimization; extra spins and couplings.

1) and consider an objective combining expected energy with entropy. Temporary interactions modify this objective while the labels are uncertain, and disappear before the original-QUBO endpoint is read out. We first derive how these interactions affect small probability perturbations. The following section introduces the complete solver that uses this mechanism and tests its partitions on real hypergraphs.

The first two catalysts act on vertices sharing a net and on that net's auxiliary variables. We add a pair term $\gamma(t)C_{\text{pair}}$ and an auxiliary term $C_{\text{aux}}(t)$, where

$$\begin{aligned} C_{\text{pair}} &= \sum_e w_e a_e \sum_{i<j\in e} z_i z_j (x_i - x_j)^2, \\ C_{\text{aux}}(t) &= -\sum_e \kappa_e(t)(u_e - v_e)^2. \end{aligned} \tag{6}$$

The binary auxiliaries u_e, v_e encode the net cost, as defined above. Positive node scales z_i redistribute pair coupling according to incidence degree and local degree contrast; a rescaling of a_e holds its total coupling mass fixed. The auxiliary term favours states in which one net auxiliary is one and the other is zero, changing the net's response to uncertain vertex labels. Both terms remain quadratic, and their amplitudes vanish at the annealing endpoints. Temporary per-spin temperatures provide a further control by changing the marginal entropy weights. These offsets also disappear before the terminal original-QUBO phase (Fig. 2).

The effect of a catalyst can be calculated from the free-energy curvature around an uncertain state. We first minimize each four-state net-auxiliary distribution at fixed vertex probabilities, obtaining a reduced free energy that depends on $\mathbf{p}$. For a fixed strict count target, even n, uniform temperature T and $p_i = 1/2$, its probability Hessian for the local pair and auxiliary terms is

$$\begin{aligned} \mathbf{H} = {} & 4T\mathbf{I} + 2\lambda(\mathbf{1}\mathbf{1}^{\mathsf{T}} - \mathbf{I}) \\ & - \mathbf{B}^{\mathsf{T}} \operatorname{diag}(w_e^2 \chi_e / T)\mathbf{B} \\ & - 2\gamma \mathbf{A}_{\text{off}}. \end{aligned} \tag{7}$$

Here $\mathbf{B}$ is net-by-vertex incidence, χ_e is the variance of the auxiliary difference $v_e - u_e$, and $\mathbf{A}_{\text{off}}$ contains the off-diagonal pair weights. The four terms respectively describe entropy, the full expected balance penalty, the response of the minimized auxiliary blocks, and the added pair interaction. Supplementary Note 3 derives each contribution. The analytic expression agrees with automatic second derivatives on weighted test instances.

Balanced perturbations preserve the expected block load, so their probability changes sum to zero. Projecting $\mathbf{H}$ onto this subspace separates such rearrangements from a uniform change in block size. Positive curvature resists a small perturbation; negative curvature lowers the free energy along it. The pair operator depends on which vertices share nets, so incidence degree alone cannot specify the affected directions. On example B, varying pair strength shifts the lowest balanced curvature as temperature decreases (Fig. 2(c)). This calculation tests how the intermediate search can change while the final binary objective remains fixed.

We introduce collective catalysts to control extended balanced directions beyond the local net couplings. Classical spectral partitioning motivates low-frequency modes of incidence-derived Laplacians[26,27]. We compute both unscaled and degree-normalized modes from the original hypergraph, and express them as zero-mean, unit-norm

vertex directions $\mathbf{v}_\ell$. Their collective binary interaction is

$$C_{\mathrm{col}}(t) = -\frac{\rho(t)}{2}\sum_\ell \theta_\ell \left[(\mathbf{v}_\ell^{\mathsf{T}}\mathbf{s})^2 - \|\mathbf{v}_\ell\|^2\right], \qquad s_i = 2x_i - 1, \tag{8}$$

with $\theta_\ell \geq 0$ and $\sum_\ell \theta_\ell = 1$. Because $\mathbf{1}^{\mathsf{T}}\mathbf{v}_\ell = 0$, a probability displacement along one direction preserves the mean vertex count. The modes enter as temporary couplings; they never supply an initial binary partition.

The collective curvature quantifies when a direction can affect the balanced response. For one mode, the exact Bernoulli expectation adds $-4\rho[\mathbf{v}\mathbf{v}^{\mathsf{T}} - \mathrm{diag}(v_i^2)]$ to the probability Hessian. Its curvature along $\mathbf{v}$ changes by $-4\rho(1-\sum_i v_i^4)$. The factor $1-\sum_i v_i^4$ is large for an extended mode and tends to zero as the mode becomes concentrated on one vertex. Thus mode localization directly limits this local effect. The mean-count penalty has zero quadratic response along a zero-mean direction, although its probability-variance contribution remains in the full Hessian. These statements concern classical variational curvature, with the derivation given in Supplementary Note 3.

Vertices with no native cut response require a separate treatment because they still contribute to balance. We retain vertices incident only to singleton nets or to no nets, and temporarily couple them along a random zero-mean signed direction. This lets their probabilities polarize without imposing a binary assignment. The coefficient again vanishes at both endpoints. The combined search uses these inactive-vertex terms with local interactions, collective modes and per-spin entropy schedules. Component contrasts supply collective directions when the cut-active support is disconnected (Supplementary Note 3).

The Hessian gives a dynamical prediction once the variational update is specified. At a stationary uncertain state with coefficients held fixed, linearization of the implicit probability update gives

$$\mathbf{J} = \mathbf{I} - \tfrac{1}{4}\mathbf{M}\mathbf{H}, \tag{9}$$

where $\mathbf{J}$ is the field-update Jacobian and the positive mobility $\mathbf{M}$ is determined by the count correction and per-vertex step sizes. Negative curvature therefore produces a growth multiplier above one in the actual update. For an initially positive balanced Hessian, a generalized eigenvalue problem also identifies the pair strength at which its lowest curvature first crosses zero. Supplementary Note 3 derives the mobility, finite-step stability condition and threshold.

Figure 3 gives a physical picture of this local response. A balanced perturbation raises some vertex probabilities and lowers others while preserving their mean count. On the cyclic example, the uncertain state lies at the bottom of a local free-energy bowl without the pair catalyst. The catalyst turns one balanced direction downhill: the same small perturbation then spreads rather than shrinks. Computed free-energy contours and the actual implicit updates show these two responses with temperature, penalty and catalyst coefficients held fixed. The spatial perturbation remains small throughout, so this test isolates local amplification rather than a transition between distant binary partitions.

Cyclic hypergraphs provide an analytic check of this prediction. For 12- and 24-vertex 3-uniform cycles, Fourier modes give the balanced curvatures and their instability temperatures (Fig. 2(d)). Predicted small-perturbation growth agrees with the implicit variational updates, and finite-difference Jacobians agree across 36 frozen configurations (Fig. 2(e),(f)). A four-vertex cyclic control has the same net size and incidence degree, yet its balanced curvature increases under the pair catalyst. The opposite responses follow from their different incidence spectra. These checks establish a local structure-dependent mechanism; determining the final cut still requires the finite searches reported next. A computed variational slice and the scope of this analysis are retained in Supplementary Note 3.

Solving catalytic QUBOs under capacity constraints

We next assess whether the construction produces useful balanced partitions on real hypergraphs. A free-energy machine (FEM) evolves vertex probabilities and four-state net-auxiliary distributions from random fields[25]. Temperature decreases during the search, and temporary interactions disappear before the terminal original-QUBO phase. An explicit CPU refinement then conditionally minimizes the same QUBO over net auxiliaries

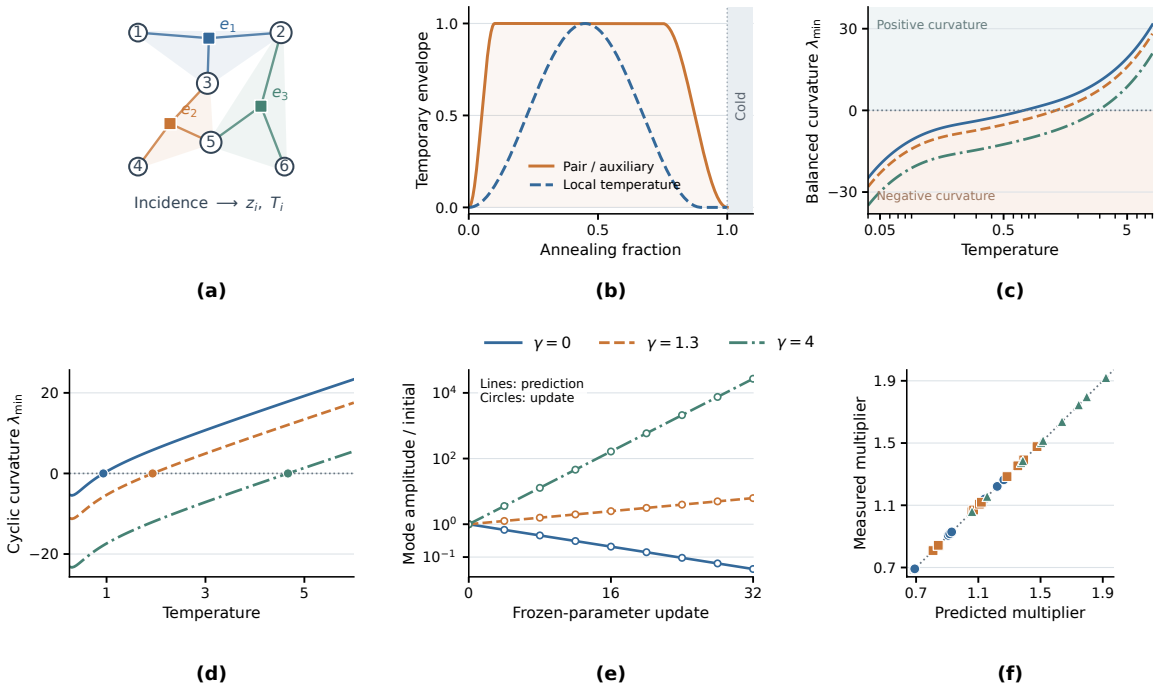


Figure 2 Catalytic response and local dynamical predictions; (a) A schematic shared-net hypergraph; squares are nets and circles are vertices, with larger circles indicating higher degree; **(b)** Temporary pair/auxiliary envelopes and local-temperature offsets vanish before the cold phase; **(c)** Computed minimum balanced curvature on example B at penalty 0.01 and pair strengths 0, 1.3 and 4; **(d)** Analytic balanced curvature on the 12-vertex cyclic 3-uniform hypergraph; dots mark the first balanced instability on cooling from $T = 6$ to 0.25; **(e)** Linear predictions (lines) and actual implicit updates (circles) for a small Fourier-mode perturbation at fixed $T = 1.5$; **(f)** Predicted and finite-difference growth multipliers across 36 configurations on 12 and 24 vertices, with and without auxiliary polarization; Panels (d)–(f) use FP32 and penalty 0.01, with protocol and proofs in Supplementary Note 3; These are local classical stability tests, not measurements of quantum gaps or global escape

and improves feasible vertex moves. Reported FEM cuts include this discrete stage; they are not attributed to variational dynamics alone. Methods specifies the complete solver.

KaHyPar supplies an independent classical reference. It contracts vertices, constructs a partition and refines it as the contractions are undone[4,5]. It receives the same original input and integer capacity, and never initializes FEM. Our direct configuration, denoted KaHyPar-K, uses one native single-thread call per trial. Supplementary Note 1 distinguishes these calls from historical repeated searches and stronger exact-balance repair pipelines.

The primary comparison covers 30 original unit-weight hypergraphs at 0%, 1% and 3% allowed imbalance, with three independent random initializations per setting. Each FEM solve uses FP32, batch size one, and a strategy selected before these evaluation runs from the earlier 48-strategy calibration. It retains every original vertex and net auxiliary, with bounded slack for the capacity bands and a complement-symmetric penalty at strict balance. Observable early stopping can shorten the maximum 3,000 annealing plus 144 terminal updates. Supplementary Note 4 records all selected strategies, calibration costs, readout controls and trial outcomes. Selection uses previously studied inputs; these measurements do not establish unseen-instance generalization.

Table 2 reports each solver's best verified native cut of three trials and its measured imbalance. Both solvers use exactly these three trials; no older winner is inserted. The FP32 pipeline improves 20 direct strict-balance references, one

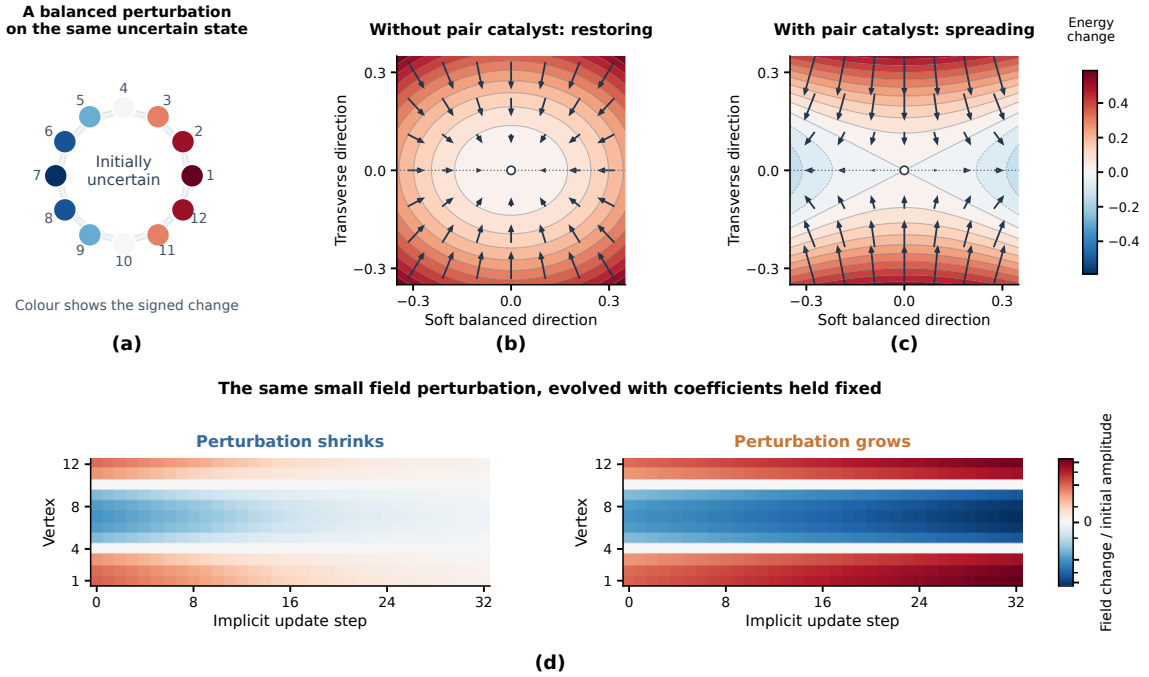


Figure 3 A temporary catalyst changes the local response of an uncertain state; (a) A small, zero-mean perturbation on a 12-vertex cyclic 3-uniform hypergraph; node colours show the signed perturbation, not a binary partition; the unperturbed vertex probabilities are all 1/2; **(b),(c)** Computed changes in reduced free energy along two orthonormal balanced probability directions, with pair strength 0 and 1.3, respectively; the same uncertain state is a local minimum without the catalyst and a saddle with it; arrows are projected probability increments from the actual implicit update, using one common scale; **(d)** Measured field perturbations over 32 updates: the initial spatial pattern decays without the catalyst and grows with it; both colour maps use the same symmetric logarithmic scale; all panels use fixed temperature 1.5, penalty 0.01, per-vertex step 0.125 and zero auxiliary and collective catalyst strengths; the initial field amplitude is 10^{-7} and field clipping is inactive; these are local classical responses, not evidence of global barrier crossing; derivations are in Supplementary Note 3

1% reference and no 3% references; the relaxed bands each have four ties. Direct strict KaHyPar runs are a limited baseline: repaired classical pipelines can achieve substantially better cuts. These win counts therefore do not establish superiority over classical hypergraph partitioning.

Discrete refinement makes a material contribution. On *scircuit*, none of the three strict-capacity CUDA trajectories has a feasible raw checkpoint, but explicit endpoint repair and refinement returns cut 164; KaHyPar reaches 84. At 1%, refinement lowers the best raw cut from 292 to 129, against KaHyPar's 46. All 270 refined FEM outputs are feasible, including fourteen strict trials without a feasible raw checkpoint. Supplementary Note 4 retains the raw and refined outcomes, together with the matched random-label refinement control. The final scores therefore measure the complete pipeline, including discrete feasibility repair and local improvement.

Historical label-symmetry controls reveal an additional search sensitivity. For odd vertex counts, replacing an oriented count penalty by a complement-symmetric penalty changes a linear probability field without changing its Hessian. Matched controls on all 12 odd inputs and one even input improve, tie and worsen six, three and three odd inputs, respectively. Their budgets and historical references remain separate from the primary FP32 table (Supplementary Note 4). The primary solver uses the symmetric encoding consistently at strict balance.

Table 2 FP32 QUBO results: best native cut of three trials and measured imbalance (%); FEM includes one preselected trajectory and explicit conditional-QUBO CPU refinement per trial. KaHyPar uses three independent direct single calls. Bold marks the lower cut, including ties. Historical FP64 and strict-capacity follow-up winners are not pooled into these cells.

	Allowed imbalance 0%		Allowed imbalance 1%		Allowed imbalance 3%	
Graph	FEM	KaHyPar	FEM	KaHyPar	FEM	KaHyPar
IBM01	383 (0.000)	**339** (0.000)	404 (0.972)	**228** (0.078)	424 (2.996)	**202** (2.760)
IBM02	448 (0.000)	**411** (0.000)	432 (0.979)	**356** (0.000)	390 (3.000)	**349** (1.418)
IBM03	**1,240** (0.000)	1,503 (0.000)	1,248 (0.994)	**963** (0.985)	1,248 (2.948)	**959** (1.893)
IBM04	909 (0.000)	**852** (0.000)	640 (0.982)	**596** (0.967)	607 (2.777)	**589** (2.218)
IBM05	2,402 (0.000)	**1,995** (0.000)	1,841 (0.607)	**1,741** (0.218)	1,831 (1.343)	**1,707** (2.821)
IBM06	**1,043** (0.000)	1,495 (0.000)	1,037 (0.683)	**1,008** (0.000)	1,046 (1.108)	**997** (2.505)
IBM07	**1,239** (0.000)	1,775 (0.000)	1,464 (0.997)	**964** (0.989)	2,231 (2.931)	**900** (2.744)
IBM08	**1,226** (0.000)	1,588 (0.000)	1,159 (0.144)	**1,140** (0.000)	1,168 (2.000)	**1,140** (0.000)
IBM09	**708** (0.000)	1,006 (0.000)	694 (0.996)	**634** (0.524)	652 (2.937)	**620** (2.663)
IBM10	**1,801** (0.000)	2,127 (0.000)	1,702 (0.994)	**1,342** (0.000)	1,733 (2.999)	**1,317** (1.564)
IBM11	**1,392** (0.000)	1,827 (0.000)	1,483 (0.921)	**1,080** (0.553)	1,309 (2.999)	**1,062** (2.891)
IBM12	**2,836** (0.000)	3,634 (0.000)	2,650 (0.999)	**2,114** (0.000)	2,764 (2.760)	**1,936** (2.476)
IBM13	2,507 (0.000)	**1,415** (0.000)	2,433 (1.000)	**849** (0.470)	2,795 (1.546)	**848** (0.404)
IBM14	**2,517** (0.000)	3,305 (0.000)	3,133 (0.902)	**1,850** (0.000)	3,452 (3.000)	**1,849** (0.000)
IBM15	**4,873** (0.000)	5,622 (0.000)	4,543 (0.999)	**2,839** (0.722)	5,974 (0.275)	**2,741** (1.868)
IBM16	5,624 (0.000)	**4,233** (0.000)	7,650 (0.892)	**2,107** (1.000)	7,602 (1.486)	**1,969** (2.887)
IBM17	**3,541** (0.000)	3,601 (0.000)	3,249 (0.999)	**2,365** (0.861)	2,716 (3.000)	**2,310** (2.600)
IBM18	**1,857** (0.000)	2,906 (0.000)	**1,922** (0.996)	1,927 (0.464)	2,356 (1.297)	**1,915** (1.878)
2cubes_sphere	**4,049** (0.000)	7,037 (0.000)	9,259 (0.623)	**3,922** (0.993)	8,062 (2.641)	**3,870** (2.995)
G2_circuit	3,977 (0.000)	**1,186** (0.000)	**624** (0.779)	**624** (0.000)	**624** (0.779)	**624** (0.000)
PGPgiantcompo	477 (0.000)	**448** (0.000)	485 (0.955)	**361** (0.000)	485 (2.397)	**352** (0.000)
airfoil_2d	**352** (0.000)	436 (0.000)	353 (0.999)	**328** (0.000)	360 (2.744)	**324** (2.899)
bcsstk29	**372** (0.000)	414 (0.000)	**360** (0.172)	**360** (0.858)	**360** (0.172)	**360** (0.858)
ca-CondMat	**4,372** (0.000)	4,764 (0.000)	4,730 (0.986)	**3,981** (0.994)	4,235 (3.000)	**3,878** (3.000)
cfd1	**2,645** (0.000)	3,243 (0.000)	2,080 (0.640)	**1,564** (0.725)	1,878 (2.887)	**1,564** (0.725)
finan512	**146** (0.000)	973 (0.000)	**146** (0.000)	**146** (0.000)	**146** (0.000)	**146** (0.000)
parabolic_fem	**1,026** (0.000)	1,836 (0.000)	**1,026** (0.014)	**1,026** (0.000)	**1,026** (0.014)	**1,026** (0.000)
scircuit	164 (0.000)	**84** (0.000)	129 (0.995)	**46** (0.894)	44 (2.498)	**42** (2.274)
shallow_water2	2,515 (0.000)	**950** (0.000)	768 (0.000)	**640** (0.000)	768 (0.000)	**640** (0.000)
thermal1	**256** (0.000)	354 (0.000)	256 (0.036)	**239** (0.711)	256 (0.036)	**239** (0.711)

CUDA execution reduces latency but leaves quality gaps

The runtime comparison measures a standalone C++/CUDA implementation of the same FP32 solver. Input and model construction, schedules and readout run without a Python interpreter. A single device-controlled CUDA Graph performs the variational updates, checkpoint statistics, early stopping and endpoint selection, followed by one final host readout. This removes repeated host intervention during the search; the conditional-QUBO CPU refinement remains part of the complete solve.

Table 3 reports the full process wall time. It gives medians of the same three trials used in Table 2, rather than time to the best-of-three cut; three serial trials instead require their summed costs. Figure 4 separates preparation, search and refinement within that complete solve. FEM uses one A100 GPU, whereas KaHyPar-K uses one local CPU thread; these measurements compare implementations on the stated resources, rather than equal hardware.

Figure 4 decomposes the complete native process time. CUDA search is now the largest component for most inputs, while initializing a CUDA context and preparing a graph remain visible fixed costs. Instances are ordered by original vertex count and capacities are kept separate. Each bar uses one actual median-time trial, so its components sum to the full FEM time in Table 3.

Table 3 Runtime per trial (s); Each entry is the median of the same three trials used in Table 2. FEM uses FP32 and includes process startup, input/model and cached-mode loading, device initialization, search, final readout and CPU refinement. KaHyPar uses one complete single-thread call. Bold marks the shorter median. Prior mode construction and policy calibration are excluded and reported in Supplementary Note 4. The original vertex count n excludes encoding auxiliaries.

		Allowed imbalance 0%		Allowed imbalance 1%		Allowed imbalance 3%	
Graph	Vertices n	FEM	KaHyPar	FEM	KaHyPar	FEM	KaHyPar
IBM01	12,752	1.514	**0.266**	1.749	**0.365**	1.788	**0.415**
IBM02	19,601	1.836	**0.515**	1.140	**0.766**	1.269	**0.867**
IBM03	23,136	1.853	**0.515**	1.876	**0.716**	1.878	**0.866**
IBM04	27,507	1.976	**0.666**	2.041	**0.766**	1.086	**1.016**
IBM05	29,347	1.854	**0.866**	1.854	**1.116**	1.854	**1.317**
IBM06	32,498	1.853	**0.918**	**1.096**	1.166	1.944	**1.367**
IBM07	45,926	1.893	**1.116**	1.911	**1.769**	**1.952**	2.871
IBM08	51,309	2.288	**1.467**	2.311	**2.118**	**1.276**	2.520
IBM09	53,395	1.946	**1.769**	1.981	**1.567**	1.976	**1.718**
IBM10	69,429	**1.970**	2.118	**1.977**	2.569	**2.043**	3.178
IBM11	70,558	**2.007**	2.169	**2.034**	2.469	**2.044**	2.821
IBM12	71,076	2.149	**2.118**	**2.165**	3.021	**2.162**	3.171
IBM13	84,199	**2.120**	3.020	**2.214**	3.822	**2.144**	3.621
IBM14	147,605	**2.399**	5.575	**2.424**	7.981	**2.519**	9.234
IBM15	161,570	**2.535**	7.329	**2.558**	13.146	**2.480**	17.366
IBM16	183,484	**2.458**	11.623	**2.467**	15.614	**2.465**	17.456
IBM17	185,495	**2.536**	7.980	**2.638**	9.233	**2.533**	9.983
IBM18	210,613	**2.716**	7.878	**2.696**	10.636	**2.614**	13.893
2cubes_sphere	101,492	**2.517**	2.620	**1.971**	8.434	**2.422**	68.882
G2_circuit	150,102	**2.946**	5.930	**0.912**	6.379	**0.912**	6.830
PGPgiantcompo	10,680	1.762	**0.164**	2.047	**0.264**	2.387	**0.265**
airfoil_2d	14,214	1.868	**0.264**	1.881	**0.365**	1.979	**0.515**
bcsstk29	13,992	1.694	**0.164**	0.703	**0.214**	0.720	**0.265**
ca-CondMat	23,133	2.062	**1.367**	2.208	**1.768**	**2.010**	5.825
cfd1	70,656	2.769	**2.368**	**2.507**	3.571	**2.610**	9.783
finan512	74,752	**0.580**	1.117	**0.623**	1.217	**0.623**	1.818
parabolic_fem	525,825	**1.491**	8.481	**2.225**	11.485	**2.293**	30.219
scircuit	170,998	2.629	**2.619**	**2.615**	2.820	**1.392**	3.121
shallow_water2	81,920	2.399	**1.317**	**0.684**	1.668	**0.692**	2.969
thermal1	82,654	2.010	**0.866**	**0.983**	1.517	**0.988**	4.222

Across instance medians, complete FEM latency is 2.01, 2.01 and 1.98 s at 0%, 1% and 3% allowed imbalance. KaHyPar takes 1.62, 1.94 and 3.04 s. Complete FEM latency is shorter on 12, 18 and 20 of the 30 inputs, respectively. The geometric means of the per-instance KaHyPar/FEM time ratios are 0.78, 1.23 and 1.82. These latency gains do not remove quality differences: faster completion and a best-of-three cut no worse than KaHyPar coincide on only nine, four and three inputs. For example, the 1% *scircuit* pipeline still returns 129 against KaHyPar's 46; its runtime is not time to that better target.

Time–quality curves expose this distinction. Figure 5 compares the aggregate response over all 30 inputs at 0% and 1% allowed imbalance in separate pairs of panels. Figure 6 gives the corresponding 3% comparison. The empirical reference for each setting is the best of the three KaHyPar-K calls; it is not a certified optimum. A target quality of 99% means a cut no larger than $\lfloor C_{\mathrm{ref}}/0.99 \rfloor$, not a demonstrated 99% success probability. Every trial enters the aggregate, with zero attained quality before a feasible output. The native graph exposes its selected partition only at completion; its internal device checkpoints are not treated as earlier host outputs.

Independent inputs can also share the GPU. In a separate eight-job pilot with prepared models and one shared CUDA context, four concurrent streams reduce GPU queue time from 14.18 to 6.29 s; including serial CPU refinement changes

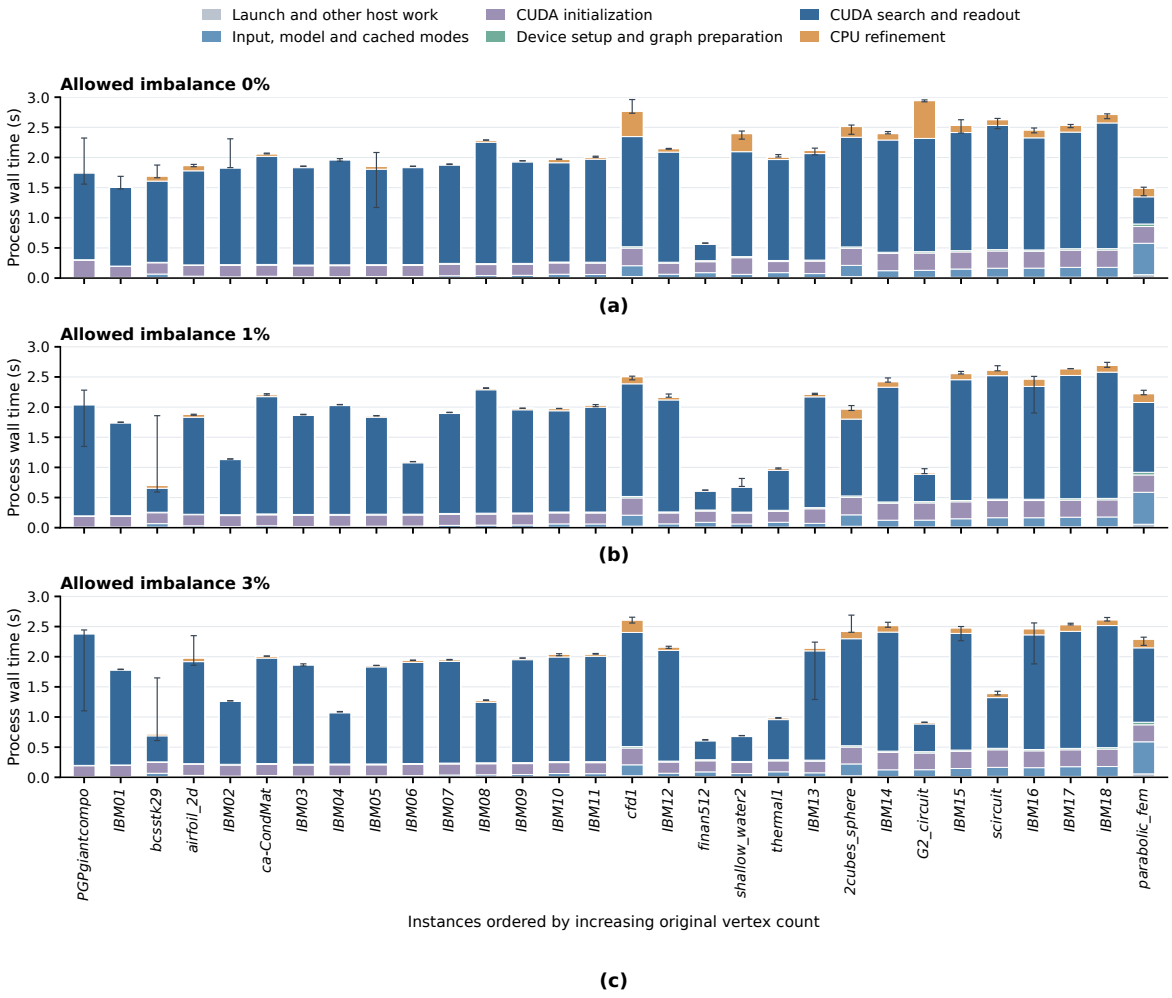


Figure 4 Native C++/CUDA FEM process time by computational stage; (a) 0%; **(b)** 1%; **(c)** 3% allowed imbalance. All 30 instances are ordered by increasing original vertex count. Each stacked bar decomposes the actual median-time trial of three, and equals the corresponding total FEM entry in Table 3; whiskers span all three total times. The six components are launch and remaining host work; input, model and cached-mode loading; CUDA initialization; device setup and graph preparation; CUDA search and final host readout; and CPU refinement. The search includes device checkpoint scoring and early stopping, with no intermediate host round trips. Prior mode construction, strategy calibration, independent auditing and archival are excluded.

the queue time from 14.52 to 6.62 s. This is a 2.19-fold throughput gain, while median active search latency increases from 1.73 to 3.02 s. These are mixed-input queue measurements, not the single-input times in Table 3, and there is no matched parallel KaHyPar throughput measurement yet (Supplementary Note 6).

Cached directions still exclude a substantial preprocessing cost. The historical construction of two incidence-mode sets exceeded 1,400 s on *parabolic_fem*; those costs must be restored when directions cannot be reused. Supplementary Note 6 retains the historical preprocessing, fusion and precision controls. An earlier Python resident-process control saved a median 1.97 s by reusing the process and GPU context. The present primary measurements instead use a fresh native executable for every trial, and include its initialization.

The current primary cohort consistently uses FP32. Historical paired FP32/FP64 controls are retained only as diagnostics, with cuts changing

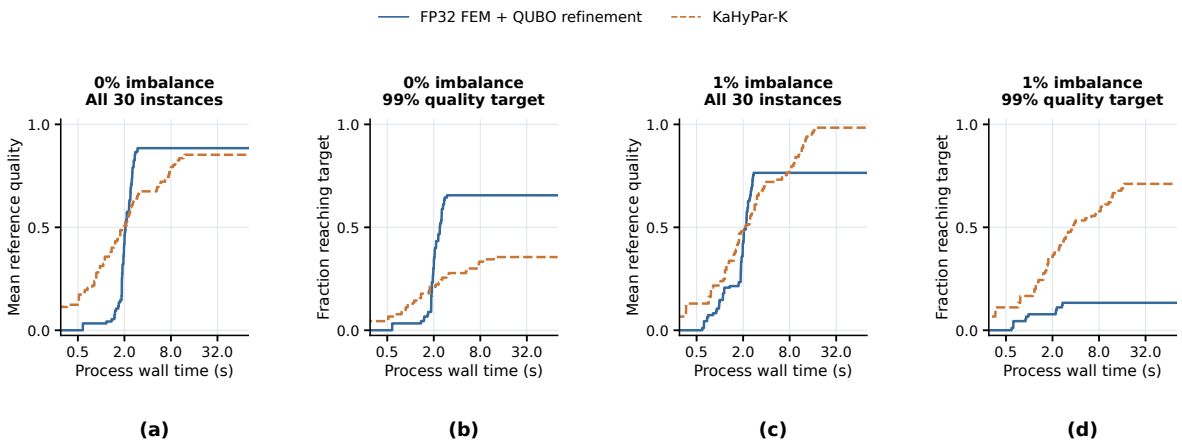


Figure 5 Time–quality response across all 30 instances at 0% and 1% imbalance; (a),(b) 0% allowed imbalance; **(c),(d)** 1% allowed imbalance; **(a),(c)** Mean attained quality, averaging $\min(1, C_{\text{ref}}/C)$ within each instance and then equally across instances, with zero before feasibility; **(b),(d)** Fraction reaching the 99% quality target with the same weighting; blue solid lines show FP32 FEM plus conditional-QUBO refinement, orange dashed lines KaHyPar-K; capacities are evaluated separately; process clocks include startup and readout, with cached modes and prior strategy calibration excluded for FEM; references are best-of-three KaHyPar cuts, not optima; FEM arrival is its returned refined partition, with no earlier arrival inferred from device checkpoint timestamps; Supplementary Note 4 gives individual-input traces

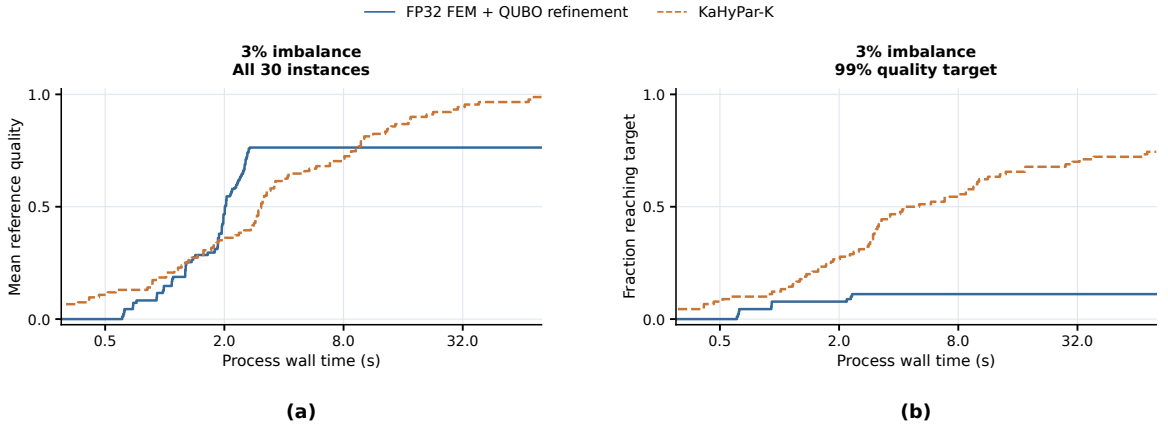


Figure 6 Time–quality response across all 30 instances at 3% imbalance; (a) Mean attained reference quality; **(b)** Fraction reaching the 99% reference-quality target; averaging, process clocks and failure accounting follow Fig. 5; all 30 inputs and 90 trials per solver enter this capacity-specific comparison

in both directions; they do not establish a general precision effect. Neither shorter execution nor local perturbation growth alone implies better final cut quality.

Discussion

The study separates preserving the cut-net objective from finding a good balanced partition of it. Exact enumeration shows that clique surrogates can prefer a larger native cut. An exact auxiliary QUBO removes that distortion, but a strong capacity penalty makes every feasible, auxiliary-minimized state a one-bit minimum under the original oriented encoding. The FEM construction addresses this search difficulty before binary readout: temporary incidence-based interactions change the response of uncertain vertex probabilities and are then removed from the final QUBO.

The quantum-inspired element is this temporary change of interactions; the algorithm and its demonstrated mechanism are classical variational dynamics.

Two local conclusions follow from the derivation. First, the reduced free-energy Hessian controls the implemented implicit update through a positive mobility: a negative curvature produces an amplifying mode, subject to the stated stationary-state and frozen-coefficient assumptions. Second, the onset of a balanced instability is set by an incidence-based generalized eigenvalue, rather than degree or net size alone. The cyclic controls demonstrate opposite catalytic responses despite matching those local statistics. These conclusions provide a calculable mechanism for path design. Predicting final cut quality from that mechanism remains an open step: local mode growth does not specify which nonlinear basin a finite annealing trajectory will reach.

The strict-capacity follow-up shows a further limit of predicting search from one local Hessian. Removing block-label bias changes a linear probability field while leaving that Hessian unchanged, yet it changes the cuts returned on several odd-sized inputs. The local instability calculation therefore explains a specific stage of FEM evolution; the rest of the annealing path also matters. The selected follow-up cuts are reported with every additional seed and unsuccessful outcome, separately from the original fixed-policy benchmark.

The real-instance results measure a complete FP32 pipeline with a preselected strategy, early stopping and explicit discrete refinement. The direct strict-balance comparison is favorable on many inputs, but quality gaps remain when capacity is relaxed and stronger classical exact-balance pipelines remain essential controls. Historical spectral rounding also reaches strong cuts on several inputs without variational evolution (Supplementary Note 5). Thus neither a good returned cut nor a favorable direct-reference comparison isolates a catalytic advantage. Mode preprocessing and instance-specific calibration are additional costs outside the cached-direction latency curves. The historical precision controls do not establish general FP32 degradation.

Overlap-gap theory concerns restrictions on specified stable algorithms for particular random ensembles[28–30]. Related restrictions apply to shallow local quantum algorithms[31,32], and recent work compares quantum and mean-field-inspired searches[33]. An overlap-gap claim for the real netlists studied here would require a specified ensemble, threshold, algorithm class and scaling analysis. Local curvature and a few returned partitions cannot determine forbidden overlaps throughout the low-cost set. Correlated variational models, including cluster mean-field[34], autoregressive distributions[35] and tensor networks[36], could test which barriers arise from independent vertex marginals. Testing transverse-field catalysts would additionally require a quantum dynamical model. For the classical solver, the next question is whether richer trial correlations change the predicted local response and improve native cut at a matched search budget.

Methods

We solve the exact cut-net QUBO in Equation (3) using a classical free-energy machine (FEM). The Results establish the encoding and the temporary interactions; here we specify how the capacity constraint, trial distribution, updates and readout form a complete solver. Supplementary Notes 2–4 provide the algebraic derivations and full numerical schedules.

Capacity penalties in the endpoint QUBO

The endpoint capacity penalty distinguishes the original benchmark from the matched follow-up. In the original strict study, block 1 has target $b = \lfloor n/2 \rfloor$ and the penalty is $\lambda(N - b)^2$, with $N = \sum_i x_i$. We use $\lambda_{\mathrm{fin}} = D_{\max} + 1$, where $D_i = \sum_{e \ni i} w_e$ and $D_{\max} = \max_i D_i$. Correcting an integer count error d costs at most $dD_{\max}$ in native cut and removes λd^2 in penalty. Thus infeasible states cannot be global minima when $\lambda > D_{\max}$ (Supplementary Note 2). Finite heuristic outputs still require an independent capacity check.

Allowed imbalance is encoded by bounded binary slack in the same QUBO. For $c = \lceil n/2 \rceil$, let $U = \lfloor (1 + \epsilon)c \rfloor$, $L = n - U$ and $S = \sum_j a_j b_j \in \{0, \ldots, U - L\}$. The balance cost is

$$E_{\mathrm{bal}} = \lambda(N - L - S)^2. \tag{10}$$

Powers of two followed by the remaining range represent every legal slack value with $\lceil \log_2(U-L+1) \rceil$ bits. Minimizing slack gives $\lambda\,\mathrm{dist}(N,[L,U])^2$ and preserves the same feasibility bound. This construction is used at 1% and 3% imbalance. The reported imbalance is $100[\max(n_0,n_1)/\lceil n/2 \rceil - 1]$.

The strict-capacity follow-up removes the arbitrary orientation of the smaller block on odd inputs. For $n = 2b+1$, its penalty is

$$\begin{aligned} P_{\mathrm{sym}}(N) &= \lambda(N-b)(N-b-1) \\ &= \lambda\big[(N-n/2)^2 - 1/4\big]. \end{aligned} \tag{11}$$

It vanishes at both legal counts and costs $\lambda d(d+1)$ at integer distance d from them, so $\lambda > D_{\max}$ remains sufficient. For even n, the centered square equals the original penalty. On odd inputs the difference from the oriented square is the linear field $\lambda(N-b)$; Supplementary Note 2 derives the complement symmetry and unchanged probability Hessian.

Free-energy machine for catalytic QUBO search

FEM searches through uncertain vertex labels before binary readout. Following mean-field annealing[37] and the FEM framework[25], we balance expected energy against entropy, cool the distribution, and increase feasibility pressure. The trial distribution, catalysts, conditional updates and schedules below belong to this one solver; replicas provide independent variational trajectories.

Trial distribution and free energy

The trial distribution specifies which correlations FEM represents. Suppressing the replica index, we use

$$q(\mathbf{x},\mathbf{u},\mathbf{v},\mathbf{b}) = \prod_i \mathrm{Bern}(x_i;p_i) \prod_e q_e(u_e,v_e) q_S(\mathbf{b}). \tag{12}$$

Vertices are independent, each net has a joint four-state auxiliary factor, and the slack bits share one joint factor when a band is used. The auxiliary blocks remain independent of the vertices. At normalized annealing time τ, the energy is

$$\begin{aligned} \mathcal{E}_\tau = E_{\mathrm{net}} &+ \lambda_\tau P_{\mathrm{bal}} + \gamma_\tau C_{\mathrm{pair}} \\ &+ C_{\mathrm{aux}}(\tau) + C_{\mathrm{col}}(\tau) + C_{\mathrm{null}}(\tau), \end{aligned} \tag{13}$$

where E_{net} is the net sum in Equation (3) and P_{bal} is the selected capacity polynomial without its coefficient. The pair, auxiliary and collective terms are defined in Results; C_{null} uses the collective form on inactive vertices. All temporary coefficients vanish at both endpoints, and $\lambda_1 = \lambda_{\mathrm{fin}}$.

The variational objective adds the negative entropies of these same factors[38]:

$$\begin{aligned} \mathcal{F}_\tau = \mathbb{E}_q[\mathcal{E}_\tau] &+ \sum_i T_i(\tau)\varphi(p_i) \\ &+ T(\tau) \sum_{e,u,v} q_e(u,v) \ln q_e(u,v) \\ &+ T(\tau) \sum_{\mathbf{b}} q_S(\mathbf{b}) \ln q_S(\mathbf{b}), \end{aligned} \tag{14}$$

where $\varphi(p) = p\ln p + (1-p)\ln(1-p)$ and the last term is omitted without slack. Uniform $T_i = T$ gives the usual variational free energy; temporary local temperatures change individual vertex entropy weights. The terminal energy is the original QUBO, while the variational updates retain the specified small terminal temperature.

Catalysts within the FEM objective

FEM evaluates the temporary interactions under the same trial distribution as the QUBO. Independence gives $\mathbb{E}[(x_i - x_j)^2] = p_i + p_j - 2p_ip_j$ for the pair term. Incidence products compute its gradient; low-rank mode products compute the collective gradients with their Bernoulli diagonal corrections. Auxiliary polarization enters the joint four-state update below. Supplementary Note 3 supplies the incidence weights, mode construction and complete derivative formulas.

Conditional auxiliary and balance updates

Before each vertex update, FEM minimizes the joint net-auxiliary factors at the current vertex probabilities. Each factor takes the Gibbs weights of its four expected state energies, giving reduced net free energy $w_e - T\ln Z_e$ and incident-vertex derivative $w_e\langle v_e - u_e\rangle$. Supplementary Note 3 gives those energies and derives the response. This update is exact within the trial family; independence between vertices and auxiliaries still limits its native-cut expectation (Supplementary Note 2).

The balance expectation retains the fluctuations of the binary labels:

$$\mathbb{E}_q[(\sum_i x_i - b)^2] = (\sum_i p_i - b)^2 + \sum_i p_i(1-p_i). \tag{15}$$

For the symmetric odd-count penalty, replace b by $n/2$ and subtract $1/4$. For a band, let g_s count the binary representations of slack value s. Its conditional weights are $q_s \propto g_s \exp[-\lambda_\tau(\mu - L - s)^2/T]$, where $\mu = \sum_i p_i$. Multiplicity preserves the binary slack entropy. The updated slack mean supplies count target $L + \mathbb{E}S$; its variance and the vertex-count variance remain in the full expectation (Supplementary Note 2).

Implicit update of the vertex probabilities

FEM represents the vertex marginals by log-odds fields $h_i = \ln[p_i/(1-p_i)]$, with $m_i = 2p_i - 1$. Let g_i be the reduced interaction derivative, including current catalysts but excluding vertex entropy and balance. The complete derivative is

$$\frac{\partial \mathcal{F}}{\partial p_i} = g_i + 2\lambda_\tau(\mu - b_{\text{eff}}) - \lambda_\tau m_i + T_i h_i. \tag{16}$$

The target b_{eff} is b for oriented strict balance, $n/2$ for symmetric strict balance, or $L + \mathbb{E}S$ after the band slack update. The variance in Equation (15) supplies the term $-\lambda_\tau m_i$.

The update treats the shared count force implicitly to damp a large terminal penalty. With $d_i = \eta_a/(1+D_i)$, $r_i = d_i/(1+2\lambda_\tau d_i)$ and $\rho_i = r_i/(1+r_i T_i)$, we use

$$\begin{aligned} \widetilde{h}_i &= \frac{h_i - r_i(g_i - \lambda_\tau m_i)}{1 + r_i T_i}, \\ h_i' &= \widetilde{h}_i - 2\lambda_\tau \rho_i \delta, \end{aligned} \tag{17}$$

where $\widetilde{h}_i$ is clipped to $[-20, 20]$ before solving

$$\delta = \sum_i \text{sigmoid}(\widetilde{h}_i - 2\lambda_\tau \rho_i \delta) - b_{\text{eff}}. \tag{18}$$

The root derivative is $1 + \sum_i 2\lambda_\tau \rho_i p_i'(1-p_i') > 0$, giving a unique solution. The primary protocol uses 64 bracketed Newton iterations; Supplementary Note 6 gives the safeguards and numerical exceptions. The solved δ is a mean-count residual, so this step applies a soft penalty rather than projecting binary labels onto an exact count.

Annealing schedules and replica allocation

Each primary solve evolves one FP32 trajectory. Its profile and local, unscaled-mode or normalized-mode geometry are fixed before evaluation, using the historical 48-strategy calibration described in Supplementary Note 4. Random fields are newly drawn for each trial; no reference partition is supplied. Historical replicas evolve independently rather than exchanging states as in exchange Monte Carlo[39].

The maximum budget is 3,000 annealing and 144 terminal updates. Temperature decreases exponentially from 8 to 0.02; the balance coefficient is 0.01 for the first 90% and rises geometrically to λ_{fin} over the final 10%. These values define the measured policy, rather than a proved optimal schedule. Temporary interactions use squared-sine envelopes with zero endpoints. Local temperatures are $T_i = T\exp[\omega(\tau)\zeta\sigma_i]$, with $\omega = 0$ from fraction 0.9 onward. Terminal updates retain $T = 0.02$, the full penalty and zero catalysts. Supplementary Note 4 gives all profiles, envelopes, selected channels and mode weights. This is classical temperature annealing[40]; entropy weights do not implement a transverse-field evolution[41].

Early stopping monitors the original cut-net objective every 64 updates. Three consecutive checks must be feasible and return the same cut, equal to the best feasible cut observed so far. This condition initiates a terminal phase with the original QUBO restored; repeated stable feasible checks then permit stopping. The maximum budget remains the fallback. A finite cut plateau indicates observable saturation rather than optimality; Supplementary Note 4 gives the terminal checks and the complete matched validation.

Binary readout and endpoint refinement

At checkpoints every 64 updates and at phase boundaries, vertex labels are $x_i = \mathbb{I}(h_i > 0)$. Net auxiliaries and slack are minimized conditionally, leaving vertex labels unchanged. The retained starting state for refinement minimizes original endpoint-QUBO energy across checkpoints, including the final balance penalty for infeasibility.

The CPU refiner first restores the integer capacity if necessary. With $\lambda_{\text{fin}} > D_{\max}$, each corrective vertex move and conditional auxiliary minimization lowers the endpoint QUBO. Once

feasible, it tests opposite-side vertex pairs and, within a capacity band, feasible single moves, using exact native-net gains. A pass may traverse nonimproving moves but retains only its best feasible prefix; the returned feasible cut cannot exceed the repaired starting cut. The measured limits are eight passes, eight candidates per side, 20,000 considered moves per pass and 512 moves without an improved prefix. These deterministic limits avoid a time-dependent final partition; all actual refinement time is charged. The limits are computational budgets, not convergence guarantees. Supplementary Note 4 compares raw FEM, refined FEM and refinement of random labels; Supplementary Note 7 specifies independent binary scoring.

Benchmark design and computational cost

Thirty original public unit-weight hypergraphs[42,43] are evaluated at 0%, 1% and 3% allowed imbalance. Each solver has three trials per setting, giving 270 FEM and 270 KaHyPar-K outputs. FEM uses one preselected strategy and random initialization per trial, with FP32 arithmetic and 64 safeguarded count-root iterations. Every checkpoint and final FEM partition is independently audited from the original net list; all classical outputs are also independently scored. The same three trials provide both tables: cut is their best observed value and time is the median individual trial cost. A best-of-three serial search instead costs the sum of its trial times.

FEM process time starts before launching the standalone native executable and ends with the host-available refined partition. It includes input, model and schedule construction, cached-mode loading, CUDA initialization, graph preparation, search, final readout and CPU refinement. The CUDA component starts immediately before graph submission and ends after its final readout and completion wait. Device statistics and stopping run within the graph; no host feedback interrupts the variational loop. Prior mode construction and strategy calibration are excluded and disclosed in Supplementary Note 4. KaHyPar-K uses one cold native CLI call on one CPU thread; only a returned partition provides an observation. Its completion time upper-bounds any unobserved internal target-discovery time. Independent audits and archival are outside both clocks. Different GPU and CPU hardware preclude an equal-hardware speedup interpretation.

Time-to-target analysis follows a fixed solution-quality criterion[44]. For positive empirical reference cut C_{ref}, target quality η sets the integer threshold $\lfloor C_{\mathrm{ref}}/\eta \rfloor$. The reference is the best of three KaHyPar-K calls for that input and capacity, fixed independently of FEM outcomes. The first returned feasible partition below the target sets arrival time; native device checkpoints do not establish host arrival; a missing hit is censored at the trial horizon. Qualities of 97% and 99% are distinct from the probability of at least one successful solve. Supplementary Note 4 gives uncertainty and restart projections; three trials do not demonstrate 97% or 99% reliability. Historical FP64 and multi-replica controls retain their own protocols and are not pooled with this primary cohort.

Data availability

Aggregate benchmark results and input provenance are available at https://github.com/FemHyPar/qubo-hypergraph-partition/tree/main/docs/benchmarks.

Code availability

The partitioning interface and protected solver releases are available at https://github.com/FemHyPar/qubo-hypergraph-partition.

References

1. Catalyurek, U. & Aykanat, C. Hypergraph-partitioning-based decomposition for parallel sparse-matrix vector multiplication. *IEEE Trans. Parallel Distrib. Syst.* **10**, 673–693 (1999). https://doi.org/10.1109/71.780863.
2. Uçar, B. & Aykanat, C. Revisiting Hypergraph Models for Sparse Matrix Partitioning. *SIAM Rev.* **49**, 595–603 (2007). https://doi.org/10.1137/060662459.
3. Karypis, G. & Kumar, V. Multilevel k -way hypergraph partitioning. In *36th ACM/IEEE Design Automation Conference* 343–348 (1999). https://doi.org/10.1145/309847.309954.

4. Fiduccia, C. & Mattheyses, R. A Linear-Time Heuristic for Improving Network Partitions. In *19th Design Automation Conference* 175–181 (1982). https://doi.org/10.1109/DAC.1982.1585498.
5. Schlag, S. et al. High-Quality Hypergraph Partitioning. *ACM J. Exp. Algorithmics* **27**, 1–39 (2022). https://doi.org/10.1145/3529090.
6. Gottesbüren, L., Heuer, T., Maas, N., Sanders, P. & Schlag, S. Scalable High-Quality Hypergraph Partitioning. Preprint at https://arxiv.org/abs/2303.17679 (2023).
7. Kadowaki, T. & Nishimori, H. Quantum annealing in the transverse Ising model. *Phys. Rev. E* **58**, 5355–5363 (1998). https://doi.org/10.1103/PhysRevE.58.5355.
8. Farhi, E. et al. A Quantum Adiabatic Evolution Algorithm Applied to Random Instances of an NP-Complete Problem. *Science* **292**, 472–475 (2001). https://doi.org/10.1126/science.1057726.
9. Albash, T. & Lidar, D. A. Adiabatic quantum computation. *Rev. Mod. Phys.* **90**, 015002 (2018). https://doi.org/10.1103/RevModPhys.90.015002.
10. Farhi, E., Goldstone, J. & Gutmann, S. A Quantum Approximate Optimization Algorithm. Preprint at https://arxiv.org/abs/1411.4028 (2014).
11. Lucas, A. Ising formulations of many NP problems. *Front. Phys.* **2**, 5 (2014). https://doi.org/10.3389/fphy.2014.00005.
12. Glover, F., Kochenberger, G. & Du, Y. Quantum Bridge Analytics I: a tutorial on formulating and using QUBO models. *4OR* **17**, 335–371 (2019). https://doi.org/10.1007/s10288-019-00424-y.
13. Boros, E. & Gruber, A. On Quadratization of Pseudo-Boolean Functions. Preprint at https://arxiv.org/abs/1404.6538 (2014).
14. Ishikawa, H. Transformation of General Binary MRF Minimization to the First-Order Case. *IEEE Trans. Pattern Anal. Mach. Intell.* **33**, 1234–1249 (2011). https://doi.org/10.1109/TPAMI.2010.91.
15. Ushijima-Mwesigwa, H., Negre, C. F. A. & Mniszewski, S. M. Graph Partitioning using Quantum Annealing on the D-Wave System. Preprint at https://arxiv.org/abs/1705.03082 (2017).
16. Hadfield, S. et al. From the Quantum Approximate Optimization Algorithm to a Quantum Alternating Operator Ansatz. *Algorithms* **12**, 34 (2019). https://doi.org/10.3390/a12020034.
17. Hen, I. & Spedalieri, F. M. Quantum Annealing for Constrained Optimization. *Phys. Rev. Applied* **5**, 034007 (2016). https://doi.org/10.1103/PhysRevApplied.5.034007.
18. Li, Y. et al. Quantum Hypergraph Partitioning. In *Q-Data '26* 50–57 (2026). https://doi.org/10.1145/3811628.3811836.
19. Ibrahim, C. et al. Quantum Hypergraph Partitioning. Preprint at https://arxiv.org/abs/2605.10623 (2026).
20. Seki, Y. & Nishimori, H. Quantum annealing with antiferromagnetic fluctuations. *Phys. Rev. E* **85**, 051112 (2012). https://doi.org/10.1103/PhysRevE.85.051112.
21. Nishimori, H. & Takada, K. Exponential Enhancement of the Efficiency of Quantum Annealing by Non-Stoquastic Hamiltonians. *Front. ICT* **4**, 2 (2017). https://doi.org/10.3389/fict.2017.00002.
22. Nutricati, L. A. et al. Enhancing the energy gap of random graph problems via XX-catalysts in quantum annealing. *Quantum Sci. Technol.* **10**, 045010 (2025). https://doi.org/10.1088/2058-9565/adf2d5.
23. Hormozi, L. et al. Nonstoquastic Hamiltonians and quantum annealing of an Ising spin glass. *Phys. Rev. B* **95**, 184416 (2017). https://doi.org/10.1103/PhysRevB.95.184416.
24. Albash, T. Role of nonstoquastic catalysts in quantum adiabatic optimization. *Phys. Rev. A* **99**, 042334 (2019). https://doi.org/10.1103/PhysRevA.99.042334.
25. Shen, Z.-S. et al. Free-energy machine for combinatorial optimization. *Nat. Comput. Sci.* **5**, 322–332 (2025). https://doi.org/10.1038/s43588-025-00782-0.
26. Zhou, D., Huang, J. & Schölkopf, B. Learning with Hypergraphs: Clustering, Classification, and Embedding. In *Advances in Neural Information Processing Systems 19* 1601–1608

(2006).

27. Spielman, D. A. & Teng, S.-H. Spectral partitioning works: Planar graphs and finite element meshes. *Linear Algebra Appl.* **421**, 284–305 (2007). https://doi.org/10.1016/j.laa.2006.07.020.

28. Gamarnik, D. & Sudan, M. Limits of local algorithms over sparse random graphs. Preprint at https://arxiv.org/abs/1304.1831 (2013).

29. Gamarnik, D. The Overlap Gap Property: a Geometric Barrier to Optimizing over Random Structures. Preprint at https://arxiv.org/abs/2109.14409 (2021).

30. Gamarnik, D., Jagannath, A. & Wein, A. S. Hardness of Random Optimization Problems for Boolean Circuits, Low-Degree Polynomials, and Langevin Dynamics. Preprint at https://arxiv.org/abs/2004.12063 (2022).

31. Farhi, E., Gamarnik, D. & Gutmann, S. The Quantum Approximate Optimization Algorithm Needs to See the Whole Graph: A Typical Case. Preprint at https://arxiv.org/abs/2004.09002 (2020).

32. Chou, C.-N., Love, P. J., Sandhu, J. S. & Shi, J. Limitations of Local Quantum Algorithms on Random Max-k-XOR and Beyond. Preprint at https://arxiv.org/abs/2108.06049 (2021).

33. Goh, M. & Müller, T. The Overlap Gap Property: Separating Quantum and Quantum-Inspired Approximate Optimization Algorithms. Preprint at https://arxiv.org/abs/2609.35131 (2026).

34. Zhang, X. et al. Extreme-Scale Ising Machines with Cluster Mean-Field Theory. Preprint at https://arxiv.org/abs/2609.22645 (2026).

35. Wu, D., Wang, L. & Zhang, P. Solving Statistical Mechanics Using Variational Autoregressive Networks. *Phys. Rev. Lett.* **122**, 080602 (2019). https://doi.org/10.1103/PhysRevLett.122.080602.

36. Schollwöck, U. The density-matrix renormalization group in the age of matrix product states. *Ann. Phys.* **326**, 96–192 (2011). https://doi.org/10.1016/j.aop.2010.09.012.

37. Bilbro, G. et al. Optimization by Mean Field Annealing. In *Advances in Neural Information Processing Systems 1* 91–98 (1988).

38. Wainwright, M. J. & Jordan, M. I. Graphical Models, Exponential Families, and Variational Inference. *Found. Trends Mach. Learn.* **1**, 1–305 (2008). https://doi.org/10.1561/2200000001.

39. Hukushima, K. & Nemoto, K. Exchange Monte Carlo Method and Application to Spin Glass Simulations. *J. Phys. Soc. Jpn.* **65**, 1604–1608 (1996). https://doi.org/10.1143/JPSJ.65.1604.

40. Kirkpatrick, S., Gelatt, C. D. & Vecchi, M. P. Optimization by Simulated Annealing. *Science* **220**, 671–680 (1983). https://doi.org/10.1126/science.220.4598.671.

41. Susa, Y. et al. Exponential Speedup of Quantum Annealing by Inhomogeneous Driving of the Transverse Field. *J. Phys. Soc. Jpn.* **87**, 023002 (2018). https://doi.org/10.7566/JPSJ.87.023002.

42. KaHyPar benchmark collection. Benchmark Hypergraphs and Detailed Experimental Results of "k-way Hypergraph Partitioning via n-Level Recursive Bisection". https://doi.org/10.5281/zenodo.30176 (2015).

43. Davis, T. A. & Hu, Y. The university of Florida sparse matrix collection. *ACM Trans. Math. Softw.* **38**, 1–25 (2011). https://doi.org/10.1145/2049662.2049663.

44. King, J., Yarkoni, S., Nevisi, M. M., Hilton, J. P. & McGeoch, C. C. Benchmarking a quantum annealing processor with the time-to-target metric. Preprint at https://arxiv.org/abs/1508.05087 (2015).